

Sessions and Cookies

Document Information

Section	13 — Sessions, Authentication & Security
Subtopic	01 — Sessions and Cookies
Estimated Duration	1.5 – 2 hours
Tags	Theory Practical TypeScript
Topics Covered	HTTP statelessness, cookie mechanics, <code>cookie-parser</code> , signed cookies, <code>express-session</code> middleware, session stores, session lifecycle

Learning Objectives — This Subtopic

- Explain why HTTP is stateless and articulate the role that cookies and sessions play in preserving state across requests.
- Configure and use `cookie-parser` to read, write, and sign cookies in a TypeScript Express application.
- Integrate `express-session` middleware with type-safe session data using module augmentation.
- Distinguish between the in-memory session store (development) and an external store such as Redis (production), and explain the trade-offs of each.
- Implement session-based state tracking — such as visit counters and flash messages — and correctly manage session termination.

- Identify common cookie and session security pitfalls and apply the correct configuration options to mitigate them.

1. HTTP Statelessness and the Problem It Creates

HTTP is a *stateless* protocol: every request the client sends to the server is entirely independent. The server processes it, returns a response, and immediately forgets the connection ever happened. There is no built-in mechanism in HTTP to say "the person who made this request is the same person who made the previous one."

This is fine for serving static documents, but almost every real web application needs continuity. A user logs in, navigates to another page, and expects the server to still know who they are. A shopping cart must persist its contents across page loads. A settings form might need to report back whether a save operation succeeded.

Two complementary mechanisms solve this: **cookies** and **sessions**.

- **Cookies** are small key-value strings stored in the browser and automatically sent back to the server on every subsequent request to the same domain. They carry data on the client side.
- **Sessions** store data on the server side and give the client only a small, opaque identifier (a session ID) via a cookie. The client never sees the session data itself.

Analogy — The Hotel Key Card

When you check into a hotel, the receptionist does not memorise your face. Instead they give you a key card (the session cookie). Every time you approach a door (a request), the card is swiped (the cookie is sent). The door's reader queries the hotel system (the session store) to verify access. The key card itself contains no personal data — it is just an identifier that points to a record held securely by the hotel.

2. Project Setup

This section builds on the TypeScript Express project structure established in earlier sections. Install the additional packages needed for this subtopic.

Run the following in your terminal:

```
npm install cookie-parser express-session
```

```
npm install --save-dev @types/cookie-parser @types/express-session
```

Your updated `package.json` dependencies should include these entries:

`package.json` (excerpt)

```
{
  "dependencies": {
    "cookie-parser": "^1.4.6",
    "express": "^4.18.2",
    "express-session": "^1.17.3"
  },
  "devDependencies": {
    "@types/cookie-parser": "^1.4.6",
    "@types/express": "^4.17.21",
    "@types/express-session": "^1.17.10",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.3.0"
  }
}
```

The working project structure for this subtopic:

```
session-demo/ |— src/ | |— app.ts ← Express app setup and middleware registration | |—
session.d.ts ← TypeScript module augmentation for session types | |— routes/ | |—
cookie.routes.ts | |— session.routes.ts |— tsconfig.json |— package.json
```

`tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true
  }
}
```

```
},
"include": ["src"]
}
```

3. Cookies with `cookie-parser`

3.1 What a Cookie Actually Is

A cookie is a string the server places into the response via the `Set-Cookie` HTTP header. The browser stores it and returns it in the `Cookie` request header on all subsequent requests to that domain. A cookie has a name, a value, and several optional attributes that control its behaviour.

Attribute	Purpose
<code>HttpOnly</code>	Cookie is inaccessible to JavaScript — only sent via HTTP. Prevents XSS theft.
<code>Secure</code>	Cookie is only sent over HTTPS. Never set to false in production.
<code>SameSite</code>	Controls cross-site request inclusion. Mitigates CSRF. Values: <code>Strict</code> , <code>Lax</code> , <code>None</code> .
<code>Max-Age</code> / <code>Expires</code>	Controls cookie lifetime. Omitting both creates a session cookie — deleted when the browser closes.
<code>Domain</code> / <code>Path</code>	Scopes the cookie to a specific domain and URL path.

3.2 Reading and Writing Plain Cookies

Stage 1 Start with a minimal example: write a cookie in one route and read it in another.

```
src/app.ts
```

```
import express from 'express';
import cookieParser from 'cookie-parser';

const app = express();

app.use(cookieParser()); // parses Cookie header into req.cookies

app.get('/set-cookie', (req, res) => {
  res.cookie('language', 'en', {
    maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days in milliseconds
    httpOnly: true,
  });
  res.send('Cookie has been set.');
```

});

```
app.get('/read-cookie', (req, res) => {
  const lang = req.cookies['language'] as string | undefined;
  if (lang) {
    res.send(`Language cookie: ${lang}`);
  } else {
    res.send('No language cookie found.');
```

});

```
app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Output:

GET /set-cookie

Response body: Cookie has been set.

Response header Set-Cookie: language=en; Max-Age=604800; Path=/; HttpOnly

GET /read-cookie (after visiting /set-cookie first)

Response body: Language cookie: en

What is happening? `cookieParser()` is an Express middleware (covered in Section 7) that reads the raw `Cookie` request header and populates `req.cookies` with a plain object. `res.cookie()` writes a `Set-Cookie` response header. The browser automatically includes the cookie in all subsequent requests to `localhost:3000`.

To delete a cookie, call `res.clearCookie('language')`. This sets the cookie's `Max-Age` to zero, instructing the browser to discard it immediately.

3.3 Signed Cookies

A plain cookie value can be tampered with by the user — they can open DevTools and change `language` from `en` to anything they like. **Signed cookies** solve this by appending a cryptographic HMAC signature to the value. If the value is modified, the signature no longer matches and Express discards it.

Key Insight: Signing does *not* encrypt the cookie value. The content is still human-readable. Signing only guarantees *integrity* — that the value has not been altered since the server wrote it. To protect confidential data, use sessions (server-side storage) rather than signed cookies.

Stage 2 Pass a secret string to `cookieParser` and use the `signed: true` option when setting the cookie. Read it back via `req.signedCookies`.

src/app.ts

```
import express from 'express';
import cookieParser from 'cookie-parser';

const COOKIE_SECRET = process.env.COOKIE_SECRET ?? 'dev-secret-change-me';

const app = express();
app.use(cookieParser(COOKIE_SECRET)); // secret enables HMAC signing

app.get('/set-signed', (req, res) => {
  res.cookie('userId', '42', {
    signed: true,      // sign with COOKIE_SECRET
    httpOnly: true,
    maxAge: 60 * 60 * 1000, // 1 hour
  });
  res.send('Signed cookie set.');
```

```
});

app.get('/read-signed', (req, res) => {
  // req.signedCookies contains verified cookies; value is false if tampered
  const userId = req.signedCookies['userId'] as string | false;

  if (userId === false) {
    res.status(400).send('Cookie signature invalid – possible tampering.');
```

```
    return;
  }
}
```

```
if (userId === undefined) {  
  res.send('No signed userId cookie present.');
```



```
  return;  
}  
  
res.send(`Verified userId: ${userId}`);  
});  
  
app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Output:

GET /set-signed

Response body: Signed cookie set.

Set-Cookie: userId=s%3A42.nFz8q...K9A; Max-Age=3600; Path=/; HttpOnly

GET /read-signed (normal flow)

Response body: Verified userId: 42

GET /read-signed (after manually editing cookie in browser to "s:99.INVALID")

Status: 400

Response body: Cookie signature invalid – possible tampering.

What is happening? `cookie-parser` prepends `s:` to the serialised value and appends a dot-separated HMAC. When reading via `req.signedCookies`, the middleware verifies the HMAC using the same secret. If verification fails, the value in `req.signedCookies` is set to `false` rather than throwing — your code must check for this explicitly.

Important: Never hardcode `COOKIE_SECRET` in source code committed to version control. Load it from an environment variable (via `process.env`) or a `.env` file using the `dotenv` package, which was covered in Section 14's deployment discussion. A short or predictable secret makes HMAC forgery trivial.

4. Server-Side Sessions with `express-session`

4.1 How Sessions Work

Instead of storing data in the cookie itself, `express-session` stores it on the server and gives the browser only a random, opaque session ID. The session ID is placed in a cookie (named `connect.sid` by default). On each subsequent request, the browser sends that ID, and the middleware uses it to look up the full session object in the configured store.

The data flow on every request:

1. Browser sends `Cookie: connect.sid=s%3A...`.
2. `express-session` decodes and verifies the session ID.
3. Middleware fetches the session object from the store (in-memory, Redis, database, etc.).
4. The session object is attached to `req.session` — your route code reads and writes it freely.
5. At the end of the request, if the session was modified, it is persisted back to the store.

4.2 Type-Safe Session Data in TypeScript

`express-session` ships with a `SessionData` interface that is intentionally open for extension. To add custom properties to `req.session` without TypeScript errors, extend that interface via module augmentation.

`src/session.d.ts`

```
// This file extends express-session's SessionData interface so that
// TypeScript understands the custom properties we store in req.session.
import 'express-session';

declare module 'express-session' {
  interface SessionData {
    userId?: number;
    username?: string;
    visitCount?: number;
    flash?: string;
  }
}
```

Key Insight: Module augmentation works because `express-session` exports `SessionData` as an interface — interfaces in TypeScript are open and can be merged. Had it been exported as a `type` alias, this pattern would not be possible. You must `import 'express-session'` (even though you do not use the import) so TypeScript treats the file as a module rather than a global script; without that import, the augmentation would break ambient declarations.

4.3 Configuring the Session Middleware

Stage 1 Register the session middleware and observe how `req.session` is now available.

`src/app.ts`

```
import express from 'express';
import session from 'express-session';

// Import the augmentation so TypeScript picks up the custom SessionData fields.
import './session.d';

const app = express();

app.use(session({
  secret: process.env.SESSION_SECRET ?? 'dev-secret-change-me',
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: false, // set to true in production (HTTPS only)
    maxAge: 30 * 60 * 1000, // 30 minutes
  },
}));

app.get('/ping', (req, res) => {
  res.json({ sessionId: req.sessionID, session: req.session });
});

app.listen(3000, () => console.log('Server on http://localhost:3000'));
```

Output:

```
GET /ping
{
  "sessionId": "4gF7mN...",
```

```

"session": {
  "cookie": {
    "originalMaxAge": 1800000,
    "expires": "2026-03-01T12:30:00.000Z",
    "secure": false,
    "httpOnly": true,
    "path": "/"
  }
}

```

What is happening? The session middleware creates a new session object for this client, stores it in the default in-memory store, and returns the session ID in a `Set-Cookie` header. The session object currently only contains the cookie metadata; custom data fields are added by your route handlers.

Configuration Option Reference

Option	Type	What It Controls
<code>secret</code>	<code>string string[]</code>	Signs the session ID cookie. Use an array to support secret rotation — the first element signs new cookies; all elements are tried for verification.
<code>resave</code>	<code>boolean</code>	Forces the session to be saved back to the store on every request, even if it was not modified. Set to <code>false</code> unless your store requires it.
<code>saveUninitialized</code>	<code>boolean</code>	Saves a new, unmodified session to the store. Set to <code>false</code> to avoid creating sessions for users who have not yet interacted with session data (e.g., unauthenticated visitors).
<code>name</code>	<code>string</code>	

		Cookie name. Defaults to <code>connect.sid</code> . Change it — revealing the session library in a cookie name provides unnecessary information to attackers.
<code>store</code>	<code>Store</code>	Where sessions are persisted. Defaults to <code>MemoryStore</code> . Replace with Redis, database, or any compatible store for production.
<code>rolling</code>	<code>boolean</code>	Resets the session cookie expiry on every response. Effectively keeps active users logged in. Defaults to <code>false</code> .

4.4 Reading and Writing Session Data

Stage 2 Write a simple visit counter that persists across requests for each unique browser session.

```
src/routes/session.routes.ts
```

```
import { Router, Request, Response } from 'express';

const router = Router();

router.get('/visit', (req: Request, res: Response) => {
  // Initialise the counter if this is the first visit in this session.
  if (req.session.visitCount === undefined) {
    req.session.visitCount = 0;
  }

  req.session.visitCount += 1;

  res.json({
    message: `You have visited this route ${req.session.visitCount} time(s) this session.`
  });
});
```

```
        sessionID: req.sessionID,
    });
});

router.get('/reset', (req: Request, res: Response) => {
    req.session.visitCount = 0;
    res.json({ message: 'Visit count reset.' });
});

export default router;
```

Output:

```
GET /visit (first request)
{ "message": "You have visited this route 1 time(s) this session.", "sessionID": "aB3..." }

GET /visit (second request, same browser)
{ "message": "You have visited this route 2 time(s) this session.", "sessionID": "aB3..." }

GET /visit (different browser / incognito – new session)
{ "message": "You have visited this route 1 time(s) this session.", "sessionID": "Xf9..." }
```

What is happening? Each browser receives a unique session ID in its cookie. The server maintains separate session objects for each ID in the store. Modifying `req.session.visitCount` automatically flags the session as "dirty" — `express-session` will persist the updated object to the store at the end of the request cycle.

4.5 Flash Messages

A flash message is data written to the session for exactly one subsequent request — typically used to pass a success or error message after a redirect, a common pattern in form handling.

Note: Production applications often use the `connect-flash` package, which manages flash message lifecycle automatically. The pattern below implements the same behaviour manually to illustrate the underlying mechanics without adding a dependency.

```
src/routes/session.routes.ts
```

```
import { Router, Request, Response } from 'express';

const router = Router();

// Simulate a form submission that sets a flash message, then redirects.
router.post('/save-settings', (req: Request, res: Response) => {
  // ... (settings processing logic would live here)
  req.session.flash = 'Settings saved successfully.';
  res.redirect('/session/dashboard');
});

router.get('/dashboard', (req: Request, res: Response) => {
  // Read and immediately clear the flash message.
  const flash = req.session.flash;
  delete req.session.flash;

  res.json({
    flash: flash ?? null,
    page: 'Dashboard',
  });
});

export default router;
```

Output:

```
POST /save-settings
302 Redirect → /session/dashboard

GET /session/dashboard (immediately after redirect)
{ "flash": "Settings saved successfully.", "page": "Dashboard" }

GET /session/dashboard (on page refresh)
{ "flash": null, "page": "Dashboard" }
```

4.6 Destroying a Session

Destroying a session removes it from the store and clears the client's session cookie. This is the correct approach for logging a user out (covered in detail in Section 13-02). The mechanism is introduced here as part of understanding the session lifecycle.

```
src/routes/session.routes.ts (addition)
```

```
router.post('/logout', (req: Request, res: Response) => {
  req.session.destroy((err) => {
    if (err) {
      res.status(500).json({ error: 'Could not destroy session.' });
      return;
    }
    // Clear the session cookie from the browser.
    res.clearCookie('connect.sid');
    res.json({ message: 'Session destroyed. You are logged out.' });
  });
});
```

Output:

POST /logout

```
{ "message": "Session destroyed. You are logged out." }
Set-Cookie: connect.sid=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT
```

What is happening? `req.session.destroy()` is asynchronous — it accepts a callback because some stores (e.g., Redis) perform a network operation to delete the record. Once the callback fires without an error, the session object in the store is gone. `res.clearCookie()` then tells the browser to discard its copy of the session cookie by setting `Expires` to a past date.

5. Session Stores

5.1 The Default In-Memory Store

When no `store` option is passed to `express-session`, it uses the built-in `MemoryStore`. Sessions are stored in a plain JavaScript object in the Node.js process heap. This works perfectly for development and testing.

Important — Never use `MemoryStore` in production. It has three fatal flaws for production workloads:

- **Memory leak:** Sessions accumulate indefinitely — the store has no built-in expiry mechanism.
- **No persistence:** Every application restart wipes all sessions, forcing all users to log in again.

- **No sharing:** If your application runs across multiple Node.js processes or servers (horizontal scaling), each process has its own isolated MemoryStore. A user whose request hits a different process than the one that created their session will find it empty.

5.2 Using an External Store (Redis Example)

Any compatible store can be plugged in via the `store` option. Redis is the most common choice because of its speed, native TTL support, and pub/sub capabilities.

The `connect-redis` package bridges `express-session` and the `ioredis` client. Install them:

```
npm install connect-redis ioredis
```

```
npm install --save-dev @types/connect-redis
```

Note: You need a running Redis instance. For local development, the quickest option is Docker:

```
docker run -d -p 6379:6379 redis:alpine
```

 This requires Docker to be installed but avoids a local Redis installation.

Stage 3 Swap the default store for Redis, keeping the rest of the session configuration identical.

src/app.ts

```
import express from 'express';
import session from 'express-session';
import RedisStore from 'connect-redis';
import { Redis } from 'ioredis';
import './session.d';

const app = express();

const redisClient = new Redis({
  host: process.env.REDIS_HOST ?? 'localhost',
  port: Number(process.env.REDIS_PORT ?? 6379),
});

redisClient.on('error', (err) => {
  console.error('Redis client error:', err);
});
```

```
const store = new RedisStore({ client: redisClient });

app.use(session({
  store, // <-- only this line changes from the MemoryStore setup
  secret: process.env.SESSION_SECRET ?? 'dev-secret-change-me',
  resave: false,
  saveUninitialized: false,
  name: 'sid', // rename away from default 'connect.sid'
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 30 * 60 * 1000,
  },
}));

app.get('/ping', (req, res) => {
  res.json({ sessionId: req.sessionID });
});

app.listen(3000, () => console.log('Server on http://localhost:3000'));
```

Output:

```
Server starts without error
Server on http://localhost:3000
```

```
GET /ping
{ "sessionId": "kQ2pM..." }
```

Redis CLI verification:

```
127.0.0.1:6379> KEYS sess:*
1) "sess:kQ2pM..."

127.0.0.1:6379> TTL sess:kQ2pM...
(integer) 1798 ← seconds remaining before expiry
```

What is happening? `connect-redis` implements the `express-session` `Store` interface. When a session is created or modified, the store serialises the session object to JSON and writes it to Redis under a key prefixed with `sess:`. It also sets a TTL on that key matching `cookie.maxAge`, so expired sessions are automatically purged by Redis without any application code involvement.

5.3 Store Comparison

Store	Package	Persistence	Multi-Process Safe	Typical Use
MemoryStore (built-in)	none	No	No	Development only
Redis	<code>connect-redis</code>	Configurable (RDB/AOF)	Yes	Production standard
PostgreSQL / MySQL	<code>connect-pg-simple</code> , <code>express-mysql-session</code>	Yes	Yes	When a Redis instance is unavailable
SQLite	<code>better-sqlite3-session-store</code>	Yes	Single process only	Low-traffic or embedded apps
MongoDB	<code>connect-mongo</code>	Yes	Yes	When the stack already uses MongoDB

6. Security Checklist for Cookies and Sessions

The following settings directly affect the security posture of your session implementation. Each must be consciously set rather than left at its default.

Setting	Secure Value	Why It Matters
<code>cookie.httpOnly</code>	<code>true</code>	Prevents JavaScript from reading the session cookie. Blocks XSS-based session theft.

<code>cookie.secure</code>	<code>true</code> (production)	Cookie is only sent over HTTPS. Prevents interception on unencrypted networks.
<code>cookie.sameSite</code>	<code>'lax'</code> or <code>'strict'</code>	Restricts cross-site cookie sending. Mitigates CSRF attacks (covered in Section 8).
Session cookie name	Rename from <code>connect.sid</code>	Revealing the session library in the cookie name assists fingerprinting attacks.
<code>secret</code> length	At least 32 random bytes	Short secrets are brute-forceable. Use <code>crypto.randomBytes(32).toString('hex')</code> to generate one.
<code>saveUninitialized</code>	<code>false</code>	Avoids creating server-side sessions for every unauthenticated visitor, reducing store bloat and attack surface.
Session regeneration	Mandatory on privilege change	Call <code>req.session.regenerate()</code> after a user logs in to prevent session fixation attacks (covered in Section 13-02).

Analogy — Keeping the Key Safe

A hotel key card left on a table in the lobby (`httpOnly: false`) can be copied by any passerby (malicious script). Sending the card in an unsealed envelope (`secure: false`) risks it being intercepted in the post (on the network). A short PIN on the card management system (`secret` too short) can be guessed by trial and error. None of these flaws affect the hotel's legitimate guests until someone exploits them.

Summary

Concept	Key Point
HTTP statelessness	Each request is independent; cookies and sessions are the two primary mechanisms for maintaining state across requests.
<code>cookie-parser</code>	Parses the <code>Cookie</code> request header into <code>req.cookies</code> ; enables signed cookies when a secret is provided.
Plain cookies	Set via <code>res.cookie()</code> ; read from <code>req.cookies</code> . Client-side storage — user can read and modify the value.
Signed cookies	HMAC-protected value; read from <code>req.signedCookies</code> . Guarantees integrity but not confidentiality — value is still visible to the user.
<code>express-session</code>	Stores session data server-side; gives the browser only a signed session ID cookie. The client never sees the session payload.
Module augmentation	<code>declare module 'express-session' { interface SessionData { ... } }</code> extends the session type safely in TypeScript.
<code>resave: false</code>	Prevents unnecessary store writes on unmodified sessions — correct default for most stores.
<code>saveUninitialized: false</code>	Avoids persisting empty sessions for every visitor; reduces store bloat and is GDPR-friendlier.
MemoryStore	

	Default store; fine for development. Leaks memory, offers no persistence, and cannot be shared across processes — never use in production.
Redis store	Production-grade; shared across processes, persistent, and natively supports TTL-based expiry.
Session destruction	<code>req.session.destroy(cb)</code> removes the session from the store; pair with <code>res.clearCookie()</code> to clear the browser cookie.
Cookie security attributes	<code>httpOnly</code> , <code>secure</code> , <code>sameSite</code> , and a strong <code>secret</code> are the minimum production requirements.

Interview Questions

1. Why is HTTP described as stateless, and what practical problem does that create for web applications?

Expected answer: HTTP does not retain any information about previous requests between the server and client — each request is processed in isolation. This means the server cannot natively know, for example, that the user who just requested `/profile` is the same person who authenticated five minutes ago. Applications solve this by using cookies to persist a session identifier and sessions to associate that identifier with server-stored user data.

2. What is the difference between a plain cookie and a signed cookie? When would you choose one over the other?

Expected answer: A plain cookie stores a raw value that the user can freely modify. A signed cookie appends an HMAC to the value using a server-side secret, so any modification invalidates the signature and Express discards the cookie. Use a signed cookie when the value

has meaning and must not be tampered with (e.g., a client-side preference), but remember that signing provides integrity, not confidentiality — the value is still visible. For sensitive data, store it in a server-side session instead.

3. What is `saveUninitialized: false` in `express-session` and why is it the recommended setting?

Expected answer: When `saveUninitialized` is `true`, a new session is saved to the store the moment it is created, even if your route code never writes anything to it. Setting it to `false` ensures a session is only persisted if it has actually been modified — typically when a user logs in. This reduces unnecessary store writes, prevents creating sessions for bots and anonymous visitors, and is generally better aligned with GDPR principles around not storing user data without cause.

4. Why should you never use the default `MemoryStore` in production?

Expected answer: Three reasons: it leaks memory because sessions accumulate without built-in expiry; it is not persistent, so all sessions are lost on application restart; and it cannot be shared across multiple Node.js processes or servers, meaning a load-balanced deployment would silently route users to processes that have no record of their session.

5. What does `req.session.destroy()` do, and why is the callback signature important?

Expected answer: It removes the session from the store and unsets `req.session`. The callback is important because many production stores (such as Redis) perform an asynchronous network operation to delete the session record. You must wait for the callback before sending a response — if you respond before the session is fully destroyed, the client might make another request that still finds the old session.

6. How do you extend `req.session` with custom properties in TypeScript without losing type safety?

Expected answer: Via module augmentation: create a `.d.ts` file that imports `express-session` and declares additional properties inside `interface SessionData`. Because `SessionData` is an open interface, TypeScript merges your additions with the library's declarations, making the custom fields available on `req.session` with full type checking.

7. Explain what `cookie.sameSite: 'lax'` does and why it matters.

Expected answer: The `sameSite` attribute controls whether a cookie is sent with cross-site requests. `'lax'` allows the cookie to be sent on top-level navigations (e.g., clicking a link from an external site) but blocks it on sub-resource requests (e.g., an image tag on a foreign

page). This mitigates many CSRF attack vectors because a malicious site cannot trigger authenticated actions on your server via cross-origin form submissions or AJAX — the session cookie is withheld. `'strict'` is more restrictive but can break legitimate OAuth and redirect flows.

8. What is a flash message, and how is it implemented using the session?

Expected answer: A flash message is short-lived data stored in the session and intended to be read exactly once on the next request — most commonly used to pass a success or error notification across a POST-redirect-GET cycle. It is implemented by writing a value to a session property (e.g., `req.session.flash = 'Saved.'`), reading it in the redirected route, and immediately deleting it so subsequent requests find it gone.

Practical Assignment — Session-Backed Preference Store

Objective: Build a small Express TypeScript application where a user can save display preferences (theme and items-per-page) into their session and retrieve them on subsequent requests. The preferences must persist across page loads (within the same session) and be clearable.

1. Initialise a new TypeScript project and install all required packages:

```
npm init -y && npm install express express-session cookie-parser
npm install --save-dev typescript ts-node-dev @types/express @types/express-session
@types/cookie-parser
```

2. Create `src/session.d.ts` that augments `SessionData` with two optional fields: `theme?: 'light' | 'dark'` and `itemsPerPage?: number`.
3. Create `src/app.ts`. Register `cookieParser` and `express-session` middleware. Use a renamed cookie (`name: 'prefs_sid'`), `resave: false`, and `saveUninitialized: false`.
4. Implement `POST /preferences`. Read `theme` and `itemsPerPage` from `req.body` (register `express.json()` middleware). Validate that `theme` is either `'light'` or `'dark'`, and that `itemsPerPage` is a positive integer. Save valid values to the session and respond with the updated preferences object.
5. Implement `GET /preferences`. Return the current session preferences, or a default object (`theme: 'light', itemsPerPage: 10`) if the session contains no preferences yet.

6. Implement `DELETE /preferences` that removes the preferences from the session (do not destroy the whole session — just delete the two properties). Respond with `{ message: 'Preferences cleared.' }`.
7. Test with a REST client or curl. Confirm that preferences set in step 4 are returned by step 5 in a subsequent request, and that they are absent after step 6.

Expected Output Sequence:

Output:

```
GET /preferences (no session yet)
{ "theme": "light", "itemsPerPage": 10 }

POST /preferences body: { "theme": "dark", "itemsPerPage": 25 }
{ "theme": "dark", "itemsPerPage": 25 }

GET /preferences (same session)
{ "theme": "dark", "itemsPerPage": 25 }

POST /preferences body: { "theme": "purple" }
Status: 400
{ "error": "theme must be 'light' or 'dark'" }

DELETE /preferences
{ "message": "Preferences cleared." }

GET /preferences (same session, after clear)
{ "theme": "light", "itemsPerPage": 10 }
```

Bonus Challenge: Add a `GET /preferences/history` endpoint that returns an array of all values the `theme` preference has held during this session (including the initial default). Store this history array in the session alongside the preference values themselves. The history should append a new entry each time the theme changes via `POST /preferences`, and the history array should not be cleared when `DELETE /preferences` is called — it persists for the lifetime of the session.